TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

A

MID-TERM REPORT

ON

"TEXT TO SPEECH SYSTEM FOR NEPALI LANGUAGE USING W2V-BERT TRANSFORMER"

SUBMITTED BY:

AASHISH RUCHAL (NCE078BCT002)

AMRIT PANDEY (NCE078BCT003)

JAGADISHWOR KARKI (NCE078BCT015)

KUSHAL POKHREL (NCE078BCT016)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

LALITPUR, NEPAL

MARCH, 2026

Page of Approval

TRIBHUVAN UNIVERSIY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"Text to Speech System For Nepali Language Using W2V-BERT Transformer"** submitted by **Aashish Ruchal**, **Amrit Pandey**, **Jagadishwor Karki**, **Kushal Pokhrel** in partial fulfillment of the requirements for the Bachelor's degree in Electronics & Computer Engineering.

.....
Supervisor
Full Name
Designation
Department of Electronics and
Computer Engineering,
National College of Engineering, IOE,
TU.

.....
External examiner
Full Name
Designation
Name of Department
Name of College / Campus, IOE, TU.

Date of approval:

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, National College of Engineering, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
National College of Engineering, IOE, TU
Lalitpur, Nepal.

Acknowledgments

We would like to thank Er. Suroj Burlakoti, our Head of Department, and our Deputy Head of Department, Er. Subash Panday for giving us this opportunity to work on our project on the topic “TEXT TO SPEECH SYSTEM FOR NEPALI LANGUAGE USING W2V-BERT TRANSFORMER”.

We are deeply grateful to our project supervisor, Er. Sagar Panta, for his continuous guidance, valuable suggestions, and constructive feedback throughout the development of this project. His expertise and encouragement greatly contributed to the successful completion of our work.

We would also like to thank our teachers for their insightful suggestions and academic support, which helped us refine our project idea and improve its overall quality.

Finally, we extend our appreciation to our classmates for their cooperation and assistance, particularly during the dataset preparation stage and the audio validation process. Their support and motivation played an important role in completing this project successfully.

Contents

Page of Approval **ii**

Copyright **iii**

Acknowledgments **iv**

List of Figures **viii**

List of Tables **ix**

List of Abbreviations **x**

1 Introduction **1**

1.1 Background 1

1.2 Problem statements 2

1.3 Objectives 2

2 Literature Review **3**

2.1 Related work 3

2.2 Related Theory 5

2.2.1 Grapheme-to-Grapheme (G2G) 5

2.2.2 Montreal Forced Aligner 8

2.2.3 Wav2Vec-BERT for Phonetic Representation 11

2.2.4 FastSpeech-2: The Framework 13

2.2.5 Conformer Architecture 14

2.2.6 FastSpeech 2 with Conformer Architecture 17

2.2.7 Variance Adaptor: The Soul of the Voice 18

2.2.8 Length Regulator and Parallel Flow 18

2.2.9 Mel-Spectrogram Decoder and Conformer Blocks . . . 19

2.2.10 HiFi-GAN Vocoder 20

3 Methodology **22**
3.1 Nepali Audio Dataset Description 22
3.1.1 Utterance Characteristics 22
3.1.2 Structural and Prosodic Properties 23
3.2 Dataset Preprocessing 23
3.2.1 Audio Preprocessing 24
3.2.2 Text Preprocessing 26
3.3 Silence Modeling 31
3.3.1 Silence Interval Analysis 31
3.3.2 Bimodal Silence Distribution 33
3.3.3 Silence Classification Strategy 33
3.3.4 Grapheme Duration Analysis 34
3.3.5 Silence Injection Correction 35

4 System Design **36**
4.1 System Architecture 36
4.1.1 Phoneme Embedding Layer 36
4.1.2 Encoder 37
4.1.3 Variance Adaptor 39
4.1.4 Duration Predictor 40
4.1.5 Length Regulator 42
4.1.6 Decoder Architecture 43
4.1.7 Mel Spectrogram Projection 43
4.1.8 PostNet Refinement 43
4.1.9 Training Strategy 44
4.1.10 Loss Functions 45
4.1.11 Dataset Preparation 46
4.1.12 Speech Generation 47

5 Evaluation and Result **49**
5.1 Training Performance and Convergence Analysis 49
5.2 Spectrogram Reconstruction Analysis 50
5.3 Subjective Evaluation: Mean Opinion Score (MOS) 51

5.3.1 Results 52

6 Conclusions **53**

7 Limitations and Future enhancement **54**

7.1 Limitations 54

7.2 Future Enhancement 54

References 56

List of Figures

2.1 Conformer encoder model architecture [1] 15
2.2 FastSpeech 2 Architecture [2] 17
2.3 HiFi-GAN Vocoder [3] 20

3.1 Text Preprocessing Pipeline for Nepali TTS. 26
3.2 silence Statistics 32
3.3 Boxplot per grapheme 35

4.1 Architecture of the Proposed Nepali TTS System. 36

5.1 Tensorboard loss 49
5.2 Ground Truth Mel-spectrogram 50
5.3 Predicted Mel-spectrogram 50

List of Tables

3.1 Nepali Audio Dataset Overview 22
3.2 Audio Format 25

4.1 Teacher Forcing Scheduling 44

5.1 MOS Rating Scale and Quality Descriptions 52
5.2 Subjective Evaluation Results Summary 52

List of Abbreviations

AI	Artificial Intelligence
ASR	Automatic Speech Recognition
BERT	Bidirectional Encoder Representations from Transformers
CNN	Convolutional Neural Network
CWT	Continuous Wavelet Transform
ESNOLA	Epoch Synchronous Non-Overlap Add
FFT	Feed-Forward Transformer
G2G	Grapheme-to-Grapheme
G2P	Grapheme-to-Phoneme
GAN	Generative Adversarial Network
GVQ	Gumbel Vector Quantization
HMM	Hidden Markov Model
MCD	Mel Cepstral Distortion
MFCC	Mel-Frequency Cepstral Coefficients
MFA	Montreal Forced Aligner
MLM	Masked Language Modeling
NLP	Natural Language Processing
PCM	Pulse Code Modulation
PER	Pronunciation Error Rate
RPQ	Random Projection Quantizer
STFT	Short-Time Fourier Transform
TTS	Text-to-Speech
XLS-R	Cross-Lingual Speech Representation

1. Introduction

1.1 Background

In today's digital-first landscape, text-to-speech (TTS) technology has become indispensable to enable seamless voice-driven interaction between humans and machines. The purpose of TTS systems is to generate human-like audible speech from digital text, serving critical roles in assistive technologies, education, entertainment, and public broadcasting. Although significant advances have been made in neural TTS for high-resource languages, low-resource languages like Nepali still lack high-quality, natural-sounding synthesis systems.

In Nepal, where Nepali is the primary language of education, government, and culture, there is a growing need for voice-over technologies that support documentaries, educational content, audiobooks, and e-learning platforms. However, most existing TTS systems produce robotic or monotonous output unsuitable for narrative use.

This project proposes the development of a high-performance TTS system suited for documentary-style voiceover in Nepali. By training in natural book reading and narration-style speech, the system will generate fluent, expressive, and culturally appropriate audio with natural prosody and pacing. Our architecture leverages modern deep learning components, including a rule-based Grapheme-to-Phoneme (G2P) converter, a context-aware wav2vec-Bert Transformer.

The resulting system will enable creators, educators, and media producers to generate professional-quality Nepali voice-overs without relying on human narrators, lowering production costs and accelerating content creation. By focusing on narrative fluency and emotional neutrality, this project aims to bridge the gap between synthetic speech and human narration in the Nepali language.

1.2 Problem statements

Despite advancements in neural TTS systems, Nepali remains a low-resource language with limited high-quality speech datasets and underdeveloped models. This presents significant challenges in creating natural, intelligible, and expressive TTS systems for Nepali language. Key challenges include:

1. Scarcity of large-scale, clean, annotated Nepali speech data
2. Lack of standardized phoneme representations and linguistic tools for Nepali
3. Under-optimized attention mechanisms in long-form or expressive synthesis

1.3 Objectives

- To develop a high-quality, natural-sounding Text-to-Speech (TTS) system for the Nepali language, specifically optimized for documentary using a context-aware w2v-Bert transformer.

2. Literature Review

Text-to-Speech (TTS) technology has evolved significantly over the years. Traditional methods such as concatenative and formant-based synthesis relied on rule-based systems and recorded speech segments. While these systems produced understandable speech, they lacked natural tone and required large databases, making them less flexible and unsuitable for multilingual or real-time use. With the rise of deep learning, models like Tacotron 2 and FastSpeech introduced neural approaches that generate mel-spectrograms from text, which are then converted to speech using neural vocoders like WaveGlow or HiFi-GAN. These systems improved speech naturalness and fluency but still faced challenges related to inference speed and language scalability.

2.1 Related work

Idachaba et al. (2014) provide a comprehensive yet simplified overview TTS synthesis, outlining the fundamental components and methodologies involved in converting written text into spoken language[4]. The paper categorizes TTS systems into two main phases: the text analysis phase, which involves linguistic preprocessing such as normalization, phoneme generation, and prosody assignment, and the speech synthesis phase, where the processed input is converted into audio signals using techniques like concatenative synthesis, formant synthesis, and statistical parametric synthesis. The authors highlight the strengths and limitations of each approach, noting that while concatenative synthesis produces high-quality output, it lacks flexibility, whereas statistical models offer better adaptability at the cost of naturalness. The study also emphasizes the importance of accurate phoneme prediction and prosody modeling in achieving intelligible and natural-sounding speech, especially for complex or low-resource languages.

Bajracharya et. al (2018)[5] present a detailed case study on developing a Nepali TTS system using the Festival concatenative synthesis platform, thoroughly exploring the challenges of working with an under-resourced language such as phoneme alignment, unit selection quality, and prosodic variability and offering practical solutions like improved phone labeling, corpus design, and voice database management to enhance naturalness and intelligibility in Nepali speech synthesis.

Aaron et. al.(2016)[6] introduce WaveNet, a deep autoregressive generative model capable of synthesizing raw audio waveforms, demonstrating its superior performance over traditional concatenative and parametric TTS approaches. Their model leverages dilated causal convolutions to efficiently capture long-range temporal dependencies in high-rate audio data, with each sample conditioned on all previous ones. Evaluated on English and Mandarin, WaveNet achieves mean opinion scores that are over 50% closer to human-level speech than the best existing systems. Moreover, it supports multi-speaker conditioning and can produce realistic music samples, showcasing its versatility across audio domains.

Défossez et.al. (2022)[7] propose a state-of-the-art neural audio codec capable of real-time, high-fidelity compression through an end-to-end streaming encoder– decoder architecture with quantized latent representations. To enhance audio quality, their design introduces a multi-scale spectrogram adversarial loss and a novel “loss balancer” to stabilize training by normalizing gradient contributions across different objectives. Furthermore, they demonstrate that lightweight Transformer models can further compress representations by up to 40% while maintaining real-time performance. Extensive subjective evaluations (MUSHRA tests) reveal that this model known as EnCodec significantly outperforms traditional baselines across speech, noisy speech, and music at both 24kHz and 48kHz sampling rates .

Chettri and Shah (2013)[8] explore a concatenative Nepali TTS framework based on the Epoch Synchronous Non-Overlap Add (ESNOLA) method, in which a speech database of phoneme units (phoneme-based segments extracted at zero- crossing points) is leveraged to synthesize flat, monotonic speech. Their pipeline begins with database creation involving pitch and amplitude normalization of recorded CVCVCV (C:Consonant, V: Vowel) syllables, ensuring consistent signal quality; this is followed by text analysis to convert Unicode Devanagari input into phonemic tokens. Finally, synthesis selects and concatenates the relevant phoneme segments to generate output waveforms. Their evaluation with human listeners revealed clear and intelligible speech, though flat prosody remained a limitation. The work demonstrates a low-cost, platform-independent approach suitable for visually impaired users and basic TTS applications in Nepali contexts.

Ren. et. al. (2019)[9] introduce FastSpeech, a non-autoregressive Transformer- based TTS model that predicts phoneme durations via distillation from an autoregressive teacher and uses a length regulator to align phoneme sequences with mel-spectrogram length. This parallel feed-forward design addresses common autoregressive limitations (slow inference, skipped or repeated words, and limited controllability) by generating mel-spectrograms 270 times faster and end-to-end speech 38 times faster than autoregressive counterprts, while maintaining high quality and allowing adjustable speaking rate.

2.2 Related Theory

2.2.1 Grapheme-to-Grapheme (G2G)

Traditional TTS pipelines employ Grapheme-to-Phoneme (G2P) conversion, transforming orthographic text into phonetic representations before acoustic modeling. For Nepali, a language written in the largely phonetic De-

vanagari script we implement a streamlined Grapheme-to-Grapheme (G2G) paradigm. In this approach, each written character (grapheme) maps directly to itself as the fundamental unit for acoustic alignment, bypassing explicit phonetic transcription.

The G2G process leverages Nepali's high orthographic transparency, where the relationship between spelling and pronunciation is more regular than in languages like English. This enables the acoustic model to internalize pronunciation rules directly from data rather than relying on predefined phonetic mappings.

Mathematical Formulation:

Given a Nepali text sequence $\mathbf{T} = \{t_1, t_2, \ldots, t_n\}$ where t_i represents atomic graphemes, the G2G function $\mathcal{G}$ is an identity mapping:

$$\mathcal{G} : t_i \to g_i \quad \text{where} \quad g_i = t_i$$

For traditional G2P with phonetic dictionary $\mathcal{P}$:

$$\mathcal{P} : t_i \to p_i \quad \text{where} \quad p_i \in \Phi$$

with Φ being the phone set.

Our acoustic model $\mathcal{A}$ is trained to maximize:

$$P(\mathbf{X} \mid \mathbf{G}) = \prod_{i=1}^{n} P(\mathbf{x}_i \mid g_i)$$

where $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots\}$ are acoustic features and $\mathbf{G} = \mathcal{G}(\mathbf{T})$.

Illustrative Example:

Consider the Nepali sentence: **"नेपाल सुन्दर देश हो।"**

G2G Processing Pipeline for Nepali TTS

1. **Input Text Normalization:**

नेपाल सुन्दर देश हो। → नेपाल सुन्दर देश हो

2. **Grapheme Segmentation:**

ने	पा	ल		सु	न्	द	र		दे	श		हो
1	2	3		4	5	6	7		8	9		10

3. **G2G Dictionary Lookup (Identity Mapping):**

Input Grapheme	G2G "Pronunciation"
ने	ने
पा	पा
ल	ल
[space]	[space]
सु	सु
न्	न्
द	द
र	र
[space]	[space]
दे	दे
श	श
[space]	[space]
हो	हो

4. **Acoustic Feature Alignment:**

[Acoustic Model Visualization Placeholder]

The model learns to map graphemes directly to acoustic features, capturing pronunciation rules from data rather than explicit rules.

Implementation

Using the Python toolkit, we generate the G2G dictionary containing all unique graphemes from the Nepali speech corpus. The process involves:

1. **Corpus Analysis:** Extract all unique graphemes from the text corpus

2. **Dictionary Generation:** Create identity mappings for each grapheme
3. **Acoustic Model Training:** Train MFA to align graphemes directly with acoustic features
4. **Validation:** Verify alignment quality through manual inspection of spectrograms

The acoustic model learns the grapheme-to-acoustic mapping in an end-to-end fashion while maintaining interpretability through forced alignment. This approach is particularly effective for Nepali due to the script's syllabic structure, where each akshara (syllabic unit) serves as a natural alignment boundary.

2.2.2 Montreal Forced Aligner

Accurate alignment between text transcripts and corresponding audio signals is essential for training high-quality TTS models. Forced alignment enables the estimation of precise temporal boundaries for linguistic units (words or phonemes) within speech recordings. In this work, Montreal Forced Aligner (MFA) is employed to perform text-audio alignment for the Nepali speech dataset.

Why Forced Alignment Is Needed?

Modern TTS architectures, particularly non-autoregressive models such as FastSpeech2, explicitly rely on duration information for each phoneme or token. Without reliable alignment, models may suffer from:

- Incorrect duration prediction
- Unstable attention or monotonicity errors
- Degraded prosody and pronunciation

Forced alignment bridges this gap by mapping each textual unit to a precise time interval in the audio signal, enabling supervised duration modeling and robust training.

Why Montreal Forced Aligner?

Montreal Forced Aligner is a widely adopted, open-source forced alignment tool based on statistical acoustic modeling. It is chosen for the following reasons:

- Proven reliability across low-resource and high-resource languages
- Support for custom dictionaries and acoustic models
- Robust performance on short utterances
- Compatibility with phoneme-level and word-level alignments

Unlike attention-based neural aligners, MFA provides deterministic and interpretable alignments, making it particularly suitable for dataset preparation and quality-controlled TTS pipelines.

Core Components of Montreal Forced Aligner

MFA operates by combining several linguistic and acoustic components:

- **Acoustic Model:** A Hidden Markov Model (HMM) trained to map audio features (e.g., MFCCs) to phonetic units.
- **Pronunciation Dictionary:** Defines the mapping between words and their phoneme sequences.
- **Text Transcripts:** Cleaned and normalized utterance-level text.
- **Audio Recordings:** Preprocessed and standardized speech signals.

Together, these components enable MFA to compute the most likely alignment between phoneme sequences and audio frames using Viterbi decoding.

Building an MFA Model for Nepali

Since pretrained MFA models are not available for Nepali, a custom MFA model is trained using the dataset itself. The model construction process involves the following steps:

1. **Phoneme Inventory Definition:** Define a set of Nepali phonemes based on Devanagari graphemes and phonological rules.
2. **Dictionary Creation:** Generate a pronunciation lexicon mapping Nepali words to phoneme sequences, either manually, via a rule-based grapheme-to-phoneme (G2P) system or grapheme-to-grapheme (G2G) system.
3. **Feature Extraction:** Extract MFCC features from audio clips, which serve as input to the acoustic model.
4. **Acoustic Model Training:** Train an HMM-based acoustic model using aligned text and audio data.
5. **Alignment Generation:** Use the trained model to produce phoneme-level time alignments for the entire dataset.

This self-trained MFA model allows accurate and language-specific alignment tailored to Nepali speech characteristics, forming a reliable foundation for duration modeling and TTS training.

Role of MFA in the TTS Pipeline

The alignments produced by MFA are used to:

- Extract phoneme durations for FastSpeech 2
- Validate text–audio consistency
- Detect misaligned or corrupted samples

By providing high-quality and interpretable alignments, MFA significantly improves training stability and output naturalness in Nepali TTS systems.

2.2.3 Wav2Vec-BERT for Phonetic Representation

Theoretical Background

Wav2Vec-BERT (W2V-BERT) is a sophisticated self-supervised framework that unifies the contrastive learning objectives of Wav2Vec 2.0 with the Masked Language Modeling (MLM) capabilities of BERT. For our Nepali TTS system, it serves as a contextual feature extractor that transforms discrete phoneme sequences into high-dimensional, context-aware acoustic-phonetic vectors.

Wav2Vec 2.0: The Acoustic Foundation

Wav2Vec 2.0 learns speech representations by solving a contrastive task. Given a raw audio signal X, the model extracts latent representations z_t through a convolutional encoder. A portion of these are masked, and the model must identify the correct quantized version q_t among a set of distractors $\tilde{Q}$. The contrastive loss $\mathcal{L}_c$ is defined as:

$$\mathcal{L}_c = -\log \frac{\exp(\text{sim}(c_t, q_t)/\kappa)}{\sum_{\tilde{q} \in \tilde{Q}} \exp(\text{sim}(c_t, \tilde{q})/\kappa)}$$

Where:

- c_t is the context vector generated by the Transformer layers.
- $\text{sim}(a, b)$ denotes the cosine similarity.
- κ is a temperature parameter that scales the distribution.

BERT: The Linguistic Contextualizer

The core of our acoustic modeling relies on W2v-BERT 2.0, a state-of-the-art foundational speech encoder. Unlike traditional encoders that rely on narrow acoustic features, W2v-BERT 2.0 acts as a "universal listener," capturing the underlying texture of human speech across 143 languages through 4.5 million hours of unlabelled audio pre-training. The BERT component introduces the Masked Language Modeling (MLM) objective. Unlike standard look-up embeddings, BERT learns a mapping $h_t = f(p_t \mid$

$p_1, \ldots, p_T)$, where the representation of a phoneme p_t is conditioned on the entire surrounding sequence. This is achieved through the Self-Attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Architectural Specifications

The model transitions from a standard Transformer to a more robust Conformer architecture, specifically designed to handle the dual nature of speech signals:

- **Hybrid Conformer Backbone:** Integrates 24 layers of Conformer blocks (approx. 600M parameters). This allows the model to capture both *local* dependencies (via Convolution) and *global* phonetic context (via Multi-Head Self-Attention).
- **Refined Quantization:** The 2.0 iteration removes the Random-Projection Quantizer (RPQ) to eliminate optimization instability. It utilizes a Gumbel Vector Quantization (GVQ) codebook to discretize continuous audio into semantic units.
- **Length Adaptor:** To bridge the gap between high-frequency acoustic frames and linguistic units, a Length Adaptor is postfixed to the encoder. This ensures the output representations are dimensionally compatible with the downstream FastSpeech 2 text decoder.

W2V-BERT Mathematical Framework

W2V-BERT combines these frameworks by using a contrastive module to "discretize" audio into a finite set of speech tokens (codebook) and a subsequent Transformer stack to perform MLM on these tokens. The joint loss function $\mathcal{L}_{total}$ during pre-training is:

$$\mathcal{L}_{total} = \alpha\mathcal{L}_c + \beta\mathcal{L}_m + \gamma\mathcal{L}_d$$

Where $\mathcal{L}_c$ is the contrastive loss, $\mathcal{L}_m$ is the masked prediction loss, and $\mathcal{L}_d$ is the diversity loss ensuring codebook utilization.

The Problem: The "Static Grapheme" Limitation

In traditional TTS systems, a grapheme such as क is always represented by the same fixed vector, regardless of its context. However, in Nepali, grapheme realization is highly context-dependent:

- **कमल (Kamal):** The grapheme क is followed by म, where the inherent vowel अ is realized as a short schwa.
- **किताब (Kitab):** The grapheme क is followed by the vowel sign ि, which alters its acoustic realization due to co-articulation effects.

As a result, treating graphemes as context-independent units fails to capture important pronunciation variations in Nepali speech synthesis.

The Solution: Contextualized Embeddings

By integrating W2V-BERT, the input to our FastSpeech 2 encoder becomes a dynamic sequence:

$$X_{input} = \text{W2V-BERT}(p_1, p_2, \ldots, p_T)$$

This allows the model to "understand" the phonetic environment. For the word कला (Kala), the representation of /l/ is mathematically informed by the preceding /k/ and following /a/ via the attention weights, leading to more natural transitions and accurate Schwa Deletion predictions.

2.2.4 FastSpeech-2: The Framework

FastSpeech 2 is a non-autoregressive text-to-speech (TTS) model, meaning it generates all mel-spectrogram frames in parallel rather than sequentially. This design makes it significantly faster than autoregressive models such as Tacotron 2.

Key Components of FastSpeech 2

- **Phoneme Encoder:** Converts input phonemes, which represent the basic sound units of Nepali, into hidden representations.
- **Variance Adaptor:** This module introduces prosodic variation into the hidden states, allowing the synthesized speech to sound more natural. It consists of:
  - **Duration Predictor:** Predicts how long each phoneme should last, which is crucial for capturing Nepali rhythm.
  - **Pitch Predictor:** Predicts the fundamental frequency (F_0) to model intonation.
  - **Energy Predictor:** Predicts the amplitude or loudness of the speech signal.
  - **Length Regulator:** Expands the phoneme-level hidden states according to predicted durations so that the sequence length matches the target mel-spectrogram length.
- **Mel-Spectrogram Decoder:** Converts the expanded hidden states into the final mel-spectrogram.

2.2.5 Conformer Architecture

The Conformer (Convolution-augmented Transformer) architecture is a neural network model designed for speech processing tasks. It combines the strengths of self-attention mechanisms from Transformer models with convolutional layers to effectively capture both long-range and local dependencies in speech signals.

The architecture was introduced in the research paper "Conformer: Convolution-augmented Transformer for Speech Recognition"[1] by Anmol Gulati and colleagues to improve speech representation learning. Unlike traditional Transformer models that primarily focus on global context, the Conformer architecture also incorporates convolution operations to model local acoustic patterns such as phoneme transitions.

A typical Conformer block consists of four main components: feed-forward modules, multi-head self-attention, a convolution module, and layer normalization with residual connections. The self-attention mechanism captures global relationships across the input sequence, while the convolution module extracts local temporal features. This combination allows the model to represent speech signals more effectively.

Due to its ability to model both global and local information, the Conformer architecture has been widely adopted in modern speech systems, including models such as Wav2Vec 2.0 and W2V-BERT.

Structure of a Conformer Block

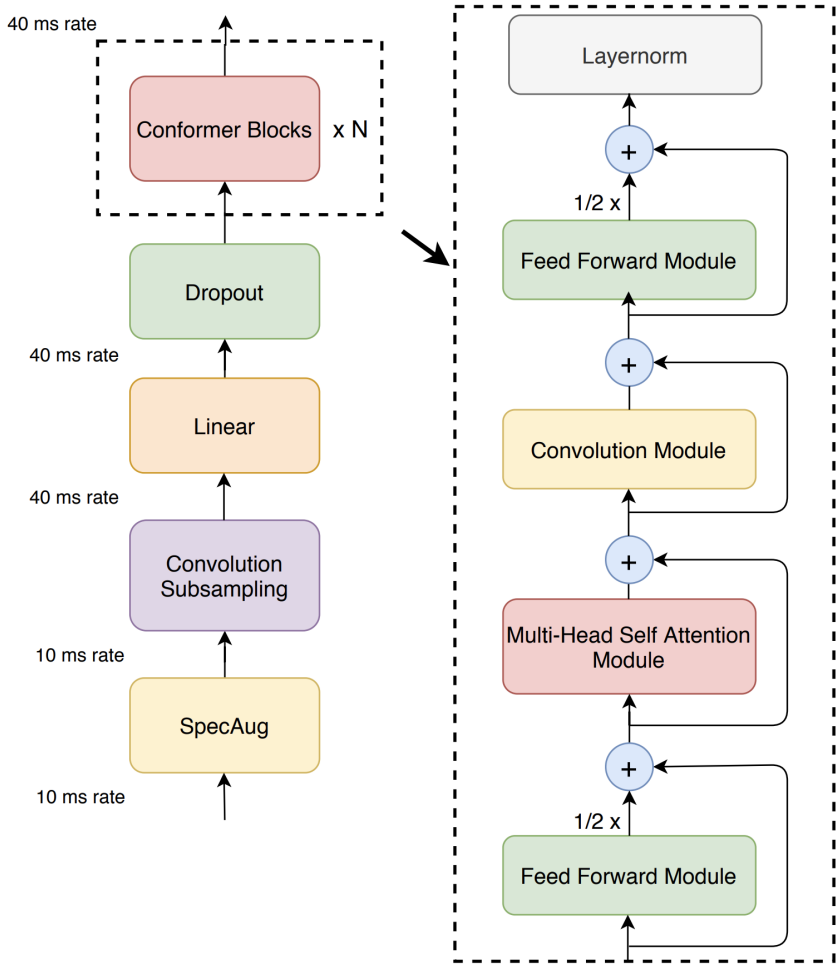

Figure 2.1: Conformer encoder model architecture [1]

A Conformer block follows a Macaron-style sandwich structure:

- **Feed-Forward Module (Half-step):** Performs initial non-linear transformation.
- **Multi-Head Self-Attention (MHSA):** Captures global relationships among all graphemes in a Nepali sentence.
- **Convolution Module:** Uses depthwise convolution to model local acoustic patterns. This is particularly effective because articulatory movements in speech exhibit strong local dependencies.
- **Feed-Forward Module (Half-step):** Applies final non-linear processing.
- **Layer Normalization:** Ensures training stability.

Mathematical Flow

Given an input representation x_i, the output y_i of a Conformer block is computed as:

$$
\begin{aligned}
\tilde{x}_i &= x_i + \frac{1}{2}\text{FFN}(x_i) \\
x'_i &= \tilde{x}_i + \text{SelfAttention}(\tilde{x}_i) \\
x''_i &= x'_i + \text{Conv}(x'_i) \\
y_i &= \text{LayerNorm}\left(x''_i + \frac{1}{2}\text{FFN}(x''_i)\right)
\end{aligned}
$$

Why Use the Conformer?

In a standard Transformer architecture, self-attention is effective at modeling global context and long-range dependencies, but it often struggles to capture fine-grained local patterns. Speech, however, is highly local, as the realization of a phoneme is strongly influenced by the immediately preceding and following sounds.

Convolutional Neural Networks (CNNs) are well suited for learning local features but are weak at modeling global structure. The Conformer architecture combines both approaches:

- **Self-Attention:** Learns the global sentence-level structure, including prosody and long-range emphasis.
- **Convolution:** Captures fine-grained local transitions between phonemes, which is essential for clear Nepali pronunciation.

Why This Matters for Nepali TTS?

- **Intelligibility:** The convolution module improves modeling of subtle consonantal distinctions, such as aspirated versus non-aspirated pairs (e.g., क vs. ख).
- **Naturalness:** Self-attention preserves sentence-level prosody, preventing robotic or word-isolated speech.
- **Efficiency:** The model remains non-autoregressive and is suitable for real-time applications.

2.2.6 FastSpeech 2 with Conformer Architecture

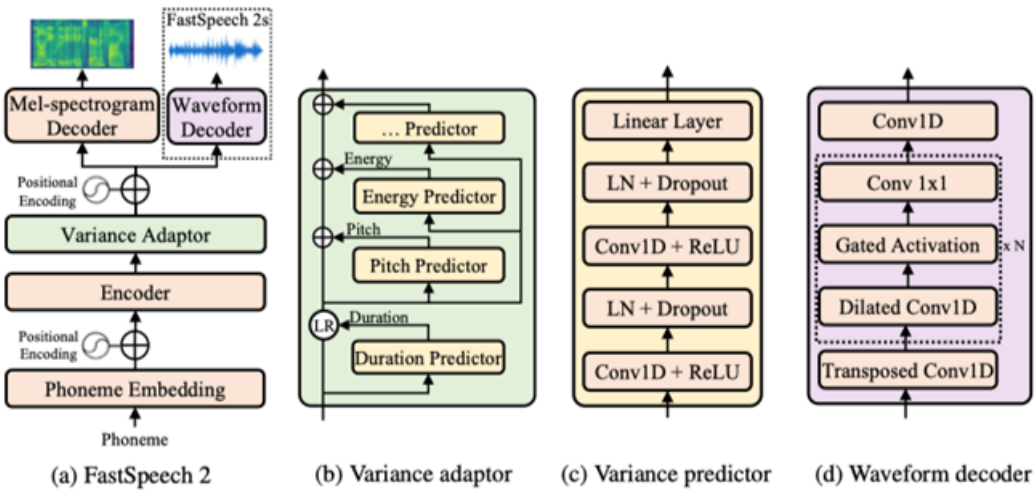

Figure 2.2: FastSpeech 2 Architecture [2]

The original FastSpeech 2 relies on Feed-Forward Transformer (FFT) blocks. While Transformers are excellent at capturing long-range dependencies , they often struggle with local relationship modeling. By replacing FFT blocks with Conformer blocks, the model gains the ability to capture both global and local features simultaneously.

Encoder Functionality

The encoder takes raw Nepali text in grapheme form and converts it into high-dimensional hidden vectors. Instead of assuming a perfect phonetic transcription, the encoder learns the relationship between written Nepali characters and their acoustic realizations directly.

This approach is particularly effective for Nepali, where pronunciation is influenced by character combinations, such as vowel-consonant interactions. The encoder effectively produces an abstract acoustic sketch of the sentence before explicit timing and pitch information are introduced.

2.2.7 Variance Adaptor: The Soul of the Voice

The Variance Adaptor addresses the one-to-many mapping problem in TTS, where the same text can be spoken at different speeds, pitches, and loudness levels. It consists of three sub-modules:

- **Duration Predictor:** Determines how long each grapheme should last. This is critical in Nepali for distinguishing between short and long vowels.
- **Pitch Predictor:** Controls sentence-level melody by predicting pitch contours, enabling natural intonation and emphasis.
- **Energy Predictor:** Predicts the intensity of each frame, allowing stressed syllables to sound more prominent.

2.2.8 Length Regulator and Parallel Flow

The Length Regulator serves as the bridge between text and audio representations. Text sequences are short, while audio representations consist of thousands of mel-spectrogram frames.

The Length Regulator expands the encoder hidden states based on predicted durations. For example, if the word नमस्कार is predicted to last 1.2 seconds, the hidden representations are stretched accordingly to match this duration.

Unlike autoregressive models that generate speech frame by frame, this process operates on the entire sentence in parallel, resulting in high inference speed.

2.2.9 Mel-Spectrogram Decoder and Conformer Blocks

The decoder converts the duration-adjusted hidden states into a mel-spectrogram. Both the encoder and decoder are constructed using Conformer blocks.

While the original FastSpeech architecture relies on FFT blocks, the Conformer-based design integrates convolutional layers. The self-attention component captures global sentence structure, while the convolution component models smooth local transitions between Nepali sounds.

Loss Functions and Optimization

To train the model effectively, several loss functions are employed.

Mel-Spectrogram Decoder Loss: The primary objective is to minimize the difference between the predicted and ground-truth mel-spectrograms. We use L1 (Mean Absolute Error) loss, which is robust to outliers and helps preserve sharp spectral details.

Variance Predictor Losses: Each variance predictor is trained with a dedicated loss function:

- **Duration Loss:** Mean Squared Error (MSE) between predicted and ground-truth durations.
- **Pitch Loss:** A Continuous Wavelet Transform (CWT)-based MSE loss, which allows modeling fine-grained pitch variations and emotional nuances.
- **Energy Loss:** MSE between predicted energy values and ground-truth energy extracted from audio using Short-Time Fourier Transform (STFT).

2.2.10 HiFi-GAN Vocoder

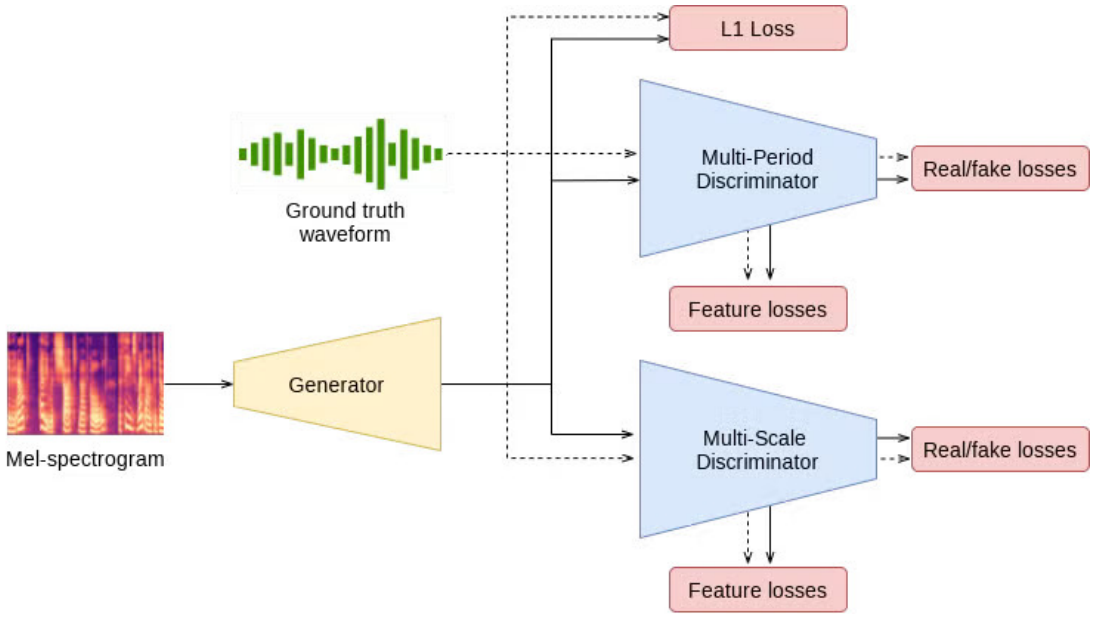

Figure 2.3: HiFi-GAN Vocoder [3]

HiFi-GAN is a neural vocoder based on generative adversarial networks (GANs). Its primary function is to convert mel-spectrograms, generated by models like FastSpeech-2, into high-quality raw audio waveforms. Traditional vocoders often struggle to achieve both high fidelity and fast inference. HiFi-GAN addresses this limitation by employing a GAN framework that includes a generator and multiple discriminators, which together model both local and global audio structures. The generator efficiently produces periodic waveform patterns, resulting in natural-sounding speech with low latency. Subjective evaluations demonstrate that HiFi-GAN can match or exceed other neural vocoders while generating audio much faster than real-time on modern GPUs.

The design of HiFi-GAN balances speed and quality: the generator uses upsampling layers to convert the low-resolution mel-spectrogram into a full waveform, while the discriminators evaluate the generated audio at multiple temporal scales to ensure both short-term detail and long-term coherence. HiFi-GAN also generalizes well to speakers not seen during training, making it suitable for multi-speaker and cross-domain synthesis.

Integration with FastSpeech 2

In a typical TTS pipeline:

- **FastSpeech-2 (Acoustic Model):** Converts input text into a mel-spectrogram that encodes pronunciation, duration, pitch, and energy information.
- **HiFi-GAN (Vocoder):** Synthesizes the final waveform from the mel-spectrogram for playback.

In this setup, FastSpeech 2 handles linguistic and prosodic modeling, while HiFi-GAN reconstructs waveform-level details, including phase information and fine temporal patterns, which are missing from the mel-spectrogram. This combination produces realistic, high-fidelity time-domain audio.

This modular approach has become standard in modern neural TTS systems because it separates text-to-acoustic modeling from acoustic-to-waveform generation, allowing each component to be optimized independently. Recent studies also explore joint training of FastSpeech-2 and HiFi-GAN to reduce mismatches between predicted and real acoustic features during inference, further improving audio quality without complex fine-tuning.

3. Methodology

3.1 Nepali Audio Dataset Description

For this research, we developed a curated Nepali speech dataset optimized for Text-to-Speech (TTS) synthesis. The primary source material consists of audiobook recordings by a single male speaker sourced from YouTube[10], ensuring high narrative quality and consistent prosody. To facilitate supervised training, the dataset includes high-fidelity manual transcriptions paired with each audio segment, stored in the uncompressed WAV format to preserve acoustic integrity. The data was meticulously processed to achieve a balance between scale, segmentation granularity, and structural coherence, rendering it suitable for both high-fidelity acoustic modeling and speaker adaptation tasks.

Table 3.1: Nepali Audio Dataset Overview

Total Duration	**10 Hours**
Number of Audio Clips	**4533**
Number of Utterances	**4533**
Average Utterance Length	**8 seconds**
Speaker Gender	**Male**
Sampling Source	**YouTube (Audiobook)**

3.1.1 Utterance Characteristics

The average duration of an utterance is approximately 8 seconds, indicating that the dataset primarily consists of short speech units, such as words or short phrases. This segmentation is particularly advantageous for non-autoregressive TTS architectures, alignment learning, and duration modeling, where shorter utterances reduce alignment ambiguity and improve training stability.

3.1.2 Structural and Prosodic Properties

The dataset leverages the expressive nature of audiobook narration, capturing a wide range of Nepali phonetic variations. By utilizing the WAV format, we ensure that no compression artifacts interfere with the feature extraction process such as Mel-spectrogram generation. The structural coherence of the recordings allows for:

- **Alignment-assisted training:** The 1-to-1 mapping of clips to utterances simplifies the learning of duration and pitch.
- **Speaker Consistency:** The use of a single-speaker source ensures stable voice characteristics across the 10-hour corpus.
- **Prosodic Naturalness:** The narrative flow of the original source provides a foundation for modeling natural speech patterns in the Nepali language.

Overall, the dataset provides a practical and structured foundation for developing and evaluating Nepali speech synthesis systems, especially when combined with pretrained representations or alignment-assisted training pipelines.

3.2 Dataset Preprocessing

Dataset preprocessing is a critical stage in building a high-quality Nepali text-to-speech (TTS) system. Neural TTS models require well-aligned and consistent audio–text pairs to learn accurate phonetic and prosodic patterns. Raw datasets often contain background noise, inconsistent recording levels, silence padding, and long recordings that are unsuitable for direct training. Therefore, preprocessing focuses on cleaning audio signals, standardizing formats, and segmenting recordings into short utterances aligned with their corresponding transcripts. Because the dataset used in this project contains long recordings (each exceeding 10 minutes), careful segmentation and alignment were required to create short, high-quality training samples. These preprocessing steps improve model stability, re-

duce training noise, and help the TTS model learn reliable text–speech correspondences.

3.2.1 Audio Preprocessing

Quality Control

Raw audio recordings may contain environmental noise, inconsistent loudness, and silence at the beginning or end of clips. Such artifacts can negatively affect the training of neural TTS systems by introducing variability that the model may incorrectly learn.

To address these issues, several quality-control steps were applied:

- **Noise Reduction:** Background noise was reduced using Audacity, applying noise profiling and spectral noise suppression to preserve speech clarity.
- **Silence Trimming:** Leading and trailing silences are trimmed using librosa to ensure tight alignment between audio and text.

  Example:

  Raw audio: [0.4s silence] "नमस्ते" [0.6s silence]
  Processed audio: "नमस्ते"

- **Volume Normalization:** All audio clips were normalized to a consistent loudness level to reduce amplitude variance across utterances.

Audio Format Standardization

Neural speech synthesis models require all audio samples to follow a consistent format. Variations in sampling rate, bit depth, or channel configuration can introduce inconsistencies in acoustic feature extraction.

Therefore, all audio files were converted to a standardized format before training.

- **Sampling Rate:** If some audio files are 20 kHz and others are 16 kHz, the model cannot learn properly. Resampling makes all audio files

Table 3.2: Audio Format

Parameter	Configuration
Sampling Rate	22.05 kHz (22050 Hz)
Bit Depth	16-bit PCM
Channels	Mono

have the same format so our audio was resampled to 22.05 kHz, a commonly used sampling rate in TTS systems because it provides a good balance between audio quality and computational efficiency.

- **Bit Depth:** 16-bit allows 65,536 amplitude levels, which captures speech clearly and has good dynamic range. So, the audio was converted to 16-bit PCM, which is widely supported by audio processing libraries and neural speech models.

- **Channels:** For TTS training we only need the voice signal, not spatial effects. Stereo also has two times more data than mono. So, Stereo recordings were converted to mono to ensure consistent acoustic representation and reduce computational cost.

This standardization ensures consistency across training samples and reduces unnecessary computational overhead.

Segmentation and Utterance Alignment

The original dataset consists of long recordings, with each audio file exceeding 10 minutes in duration. Training TTS models on long continuous recordings is inefficient and makes alignment between text and speech difficult.

To address this, the recordings were segmented into shorter clips ranging from 3 to 12 seconds using audacity. Each clip was then aligned with its corresponding transcript at the utterance level.

Example:

Original audio: "मेरो नाम राम हो। म नेपालबाट हुँ।"

Utterance-level split:

1. “मेरो नाम राम हो।”
2. “म नेपालबाट हुँ।”

3.2.2 Text Preprocessing

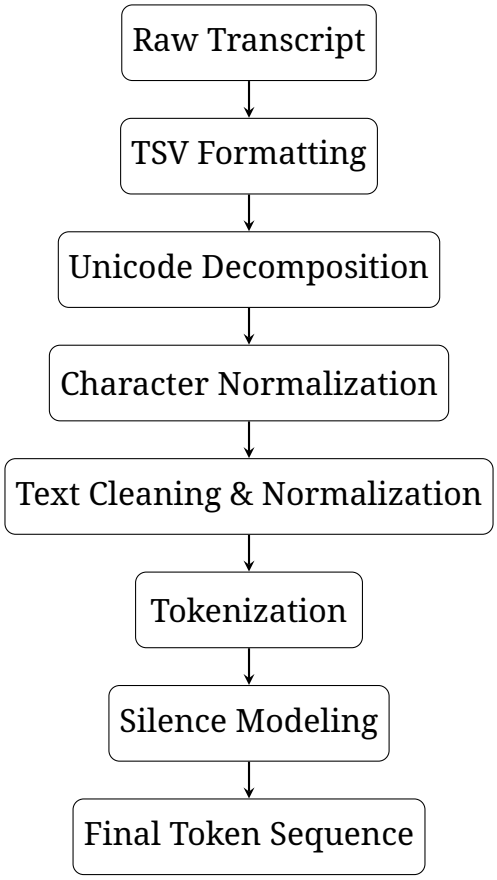

Figure 3.1: Text Preprocessing Pipeline for Nepali TTS.

Text preprocessing converts raw Nepali transcripts into a structured representation suitable for neural TTS modeling. Because TTS systems rely on accurate alignment between textual units and acoustic features, the text must be normalized, cleaned, and tokenized in a way that preserves pronunciation while minimizing ambiguity.

Transcript Formatting and Alignment

Transcript formatting and alignment were performed to establish a clear correspondence between audio clips and their textual transcripts. The

original transcripts were converted into a TSV (Tab-Separated Values) format, which provides a structured representation of the dataset. In this format, each row contains two fields: an audio identifier and its corresponding transcript.

Examples:

audio_id	**transcript**
audio_0001	मेरो नाम राम हो
audio_0002	म नेपालबाट हुँ

Each transcript directly corresponds to the segmented audio clip generated during the audio preprocessing stage. This structured format facilitates efficient dataset loading during model training and simplifies subsequent preprocessing and data management tasks.

Unicode Decomposition

Nepali text written in the Devanagari script often contains compound characters formed by combining base characters with vowel signs and joined consonant letters. These complex characters can create inconsistencies during text processing and model training. To address this issue, the transcripts were converted into a decomposed Unicode format, where compound characters are separated into their individual constituent graphemes. This transformation helps ensure consistent representation of characters at the grapheme level, which is important for speech synthesis models.

Example:

Original text: "यस पुस्तकका लेखक हुन् जेम्स"
Decomposed form: "य स प ु स ् त क क ा ल े ख क ह ु न ् ज े म ् स"

This decomposition provides several advantages for the Text-to-Speech pipeline:

- It enables grapheme-level alignment between textual units and acoustic features during model training.
- It reduces ambiguity in character representation, ensuring that similar characters are consistently processed.
- It simplifies tokenization and vocabulary construction, making it easier for the model to learn the relationship between text and speech.

Character Normalization

Character normalization was performed to reduce inconsistencies in the representation of Devanagari characters within the transcripts. In Nepali speech, certain characters may appear visually different in writing but produce very similar or identical pronunciations. If these variations are kept in the dataset, the model may treat them as different symbols even though they correspond to the same sound. To address this issue, such characters were mapped to a common or canonical form.

Examples of the applied normalization rules include:

Vowel normalization:
'ी'" → "'ि'
'ू'" → "'ु'
'ई'" → "'इ'
'ऊ'" → "'उ'
Normalization of "sa" variants:
'श'" → "'स'
'ष'" → "'स'

These mappings help reduce pronunciation variability in the dataset and prevent the model from learning multiple representations for acoustically similar sounds. This step simplifies the character space and allows the model to focus on learning meaningful speech patterns.

Character normalization is particularly useful in low-resource datasets, where the limited amount of training data may otherwise lead to inconsistent pronunciation learning if multiple character variants are present.

Text Cleaning

The transcripts were cleaned to remove non-linguistic artifacts and inconsistencies in the text. This process involved removing unnecessary punctuation symbols that do not contribute to speech generation. Unicode normalization was also applied to ensure consistent encoding of Devanagari characters across all transcripts. In addition, abbreviations and shorthand forms were expanded into their full textual representations so that the text more accurately reflects the way the sentences should be spoken during speech synthesis.

Example:

Before: "डा. शर्मा यहाँ छन्!"
After: "डाक्टर शर्मा यहाँ छन्"

Cleaning ensures that the model learns only meaningful linguistic patterns and avoids noise introduced by irregular textual symbols.

Text Normalization

Text normalization is the process of converting structured expressions in text, such as numbers, dates, and abbreviations, into their corresponding spoken forms. This step is important for Text-to-Speech systems because neural TTS models generate speech based on textual tokens. If such expressions remain in their original symbolic or numeric form, the model may not learn their correct pronunciation during training.

Examples:

Numerals: "२५" → "पच्चीस"
Numerals: "२०८०/१२/२५"" → "दुई हजार अस्सी डिसेम्बर पच्चीस"

By transforming these elements into their spoken equivalents, text normalization ensures consistent pronunciation across the dataset and improves the overall quality and generalization capability of the synthesized speech.

Tokenization

After completing the cleaning and normalization steps, the transcripts were transformed into sequences of discrete tokens that can be processed by neural Text-to-Speech models. Tokenization converts the text into smaller units, allowing the model to learn the relationship between textual symbols and their corresponding acoustic representations. In this work, the transcripts were primarily tokenized at the grapheme level to capture the structure of the Nepali language more effectively.

In addition to standard grapheme tokens, several special tokens were introduced to explicitly represent structural and acoustic events in the dataset. These tokens help the model understand sentence boundaries, pauses, and other non-verbal elements that may occur in speech.

Special tokens used in the dataset:

<pad>	Represent padding
<unk>	Unknown or out-of-vocabulary token
<SIL_LONG>	Represent long silence
<SIL_SHORT>	Represent short silence

These tokens help the model understand sentence boundaries, pauses, and other non-verbal elements that may occur in speech.

Example:

Raw sentence: मेरो नाम राम हो । म नेपालबाट हुँ
Tokenized form:
मेरो नाम राम हो <SIL_SHORT> म नेपालबाट हुँ

Such explicit structural markers are particularly beneficial for modern neural Text-to-Speech architectures, including non-autoregressive mod-

els, as they improve the alignment between textual tokens and the corresponding audio segments.

3.3 Silence Modeling

Silence is an important component of natural speech, as it contributes to the rhythm, pacing, and overall prosody of spoken language. In Text-to-Speech systems, improper handling of silence may result in unnatural or robotic speech, incorrect timing between words or phrases, and unstable alignment during model training. Therefore, it is important to carefully analyze and model silence within the speech dataset.

Silence influences several aspects of the speech generation process. It affects the monotonic alignment between text tokens and acoustic frames, which is necessary for stable model training. It also impacts duration prediction, as pauses between words or phrases determine how long certain segments of speech should be sustained. In addition, silence plays a role in identifying phrase boundaries, helping the model learn where natural pauses occur in spoken language.

Furthermore, silence contributes to the prosodic rhythm of speech, which includes the timing and flow of spoken sentences. In flow-based neural architectures, such as certain neural TTS models, the proper representation of silence also helps maintain likelihood stability during training. For these reasons, the distribution and duration of silence segments in the dataset were analyzed to ensure that the training data accurately reflects natural speech patterns.

3.3.1 Silence Interval Analysis

An analysis of silence intervals on the processed audio recordings identified 105,328 silence intervals from the dataset. These intervals represent pauses occurring between words, phrases, or sentences during natural speech.

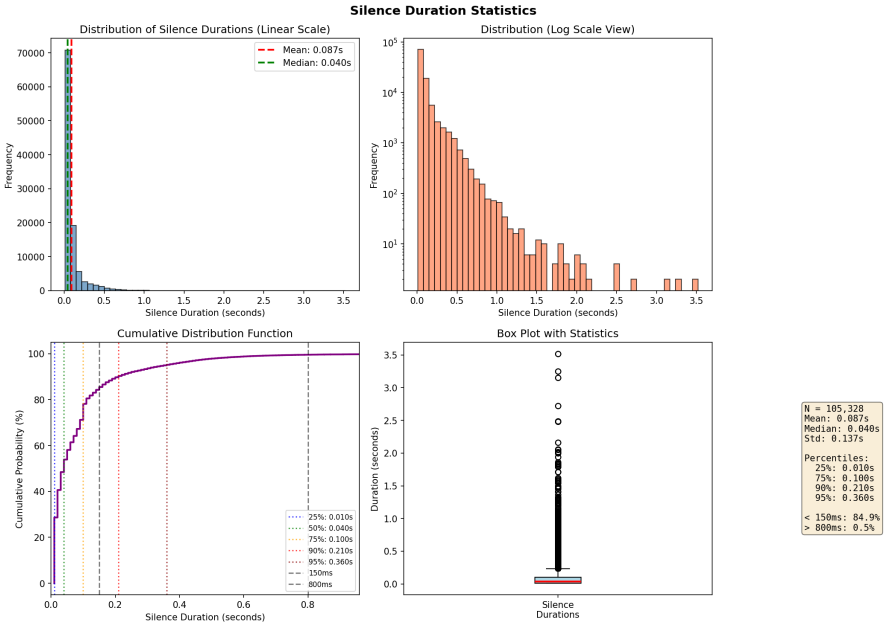

Figure 3.2: silence Statistics

Metric	**Value (s)**	**Value (ms)**
Mean	0.0875	87.5
Median	0.0400	40.0
Standard Deviation	0.1371	137.1
Range	0.0100–3.5200	10–3520

The statistical results indicate that most pauses in the dataset are relatively short. The mean silence duration is approximately 87.5 ms, while the median duration is 40 ms, suggesting that the distribution contains a large number of very short pauses. The standard deviation of 137.1 ms reflects the presence of some longer pauses that increase the variability of the distribution.

Further analysis shows that the distribution of silence durations is heavily skewed toward shorter pauses, with approximately 73% of the silence intervals being shorter than 100 ms. This observation suggests that most pauses correspond to brief gaps between words or phonetic segments, while longer pauses likely represent phrase or sentence boundaries in natural speech.

3.3.2 Bimodal Silence Distribution

Statistical analysis of the silence intervals reveals a bimodal distribution, confirming two distinct categories of pauses within the Nepali speech dataset. These clusters are modeled as a Gaussian Mixture Model (GMM):

$$p(s) = \pi_1 \mathcal{N}(\mu_1, \sigma_1) + \pi_2 \mathcal{N}(\mu_2, \sigma_2)$$

The identified clusters and their statistical properties are summarized in the table below:

Cluster (Classification)	**Weight (π)**	**Mean Duration (μ)**
Cluster 1 (Micro-pauses)	80.9%	42 ms (0.042 s)
Cluster 2 (Phrase-level)	19.1%	280 ms (0.280 s)

- **Cluster 1 (Micro-pauses):** It represents short-duration silences which occur within phonetic segments or between closely connected words.
- **Cluster 2 (Phrase-level pauses):** It corresponds to phrase-level pauses, typically occurring at word or clause boundaries, contributing to natural prosodic phrasing.

By mathematically modeling these two states, the TTS acoustic model can more accurately predict temporal spacing. This distinction is vital for generating synthesized speech that preserves the rhythmic and prosodic nuances inherent to natural Nepali conversation.

3.3.3 Silence Classification Strategy

Based on the observed bimodal distribution of silence intervals, silence durations in the dataset were categorized into three distinct groups to guide how they are represented during model training. The classification criteria are summarized below:

Silence Duration (s)	**Processing Action**
s<60 ms	Merged with neighboring phoneme
60ms=<s<300	Labeled as `<SIL_SHORT>` token
s>=300ms	Labeled as `<SIL_LONG>` token

Very short pauses, defined as durations less than 60 ms, are treated as phonetic artifacts. These are merged with the durations of adjacent phonemes rather than being modeled as independent tokens. This approach prevents the model from mistakenly interpreting micro-pauses as meaningful acoustic events, which could otherwise degrade speech naturalness

Pauses in the intermediate range (60–300 ms) are represented with a <SIL_SHORT> token to capture brief word-level or intra-phrase silences. Longer pauses, greater than 300 ms, are assigned the <SIL_LONG> token, reflecting phrase-level or sentence-boundary silences. This classification strategy ensures that the model can accurately learn both short and long silences, improving prosody and alignment in synthesized speech.

3.3.4 Grapheme Duration Analysis

To evaluate the quality of text–audio alignment, the duration of each grapheme in the dataset was analyzed using forced alignment outputs generated by the Montreal Forced Aligner (MFA). This analysis provides insight into whether grapheme-level alignment correctly corresponds to the acoustic segments of speech.

The analysis revealed that certain graphemes occasionally absorb unusually long silence intervals, which can lead to alignment drift and inconsistencies in the training data. A subset of grapheme duration statistics is presented below:

Grapheme	**Mean (ms)**	**Median (ms)**	**95th Percentile (ms)**
आ (aa)	105.8	110.0	170.0
प (pa)	95.1	100.0	160.0
स (sa)	87.9	90.0	130.0
उ (u)	96.5	90.0	160.0

While most graphemes show durations consistent with natural speech, some tokens exhibit extremely long durations (e.g., greater than 2000 ms), indicating that silence segments were incorrectly merged with phoneme

durations. Identifying and correcting these anomalies is crucial to maintain accurate grapheme-to-acoustic alignment, which directly affects the naturalness and timing of synthesized speech.

3.3.5 Silence Injection Correction

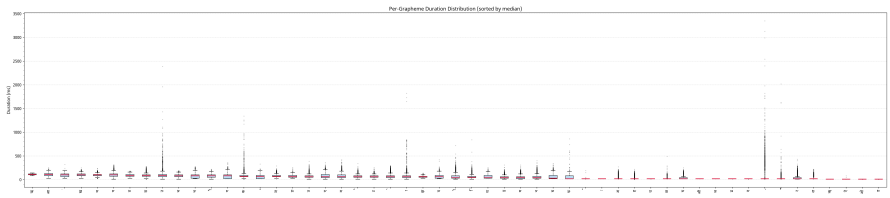

Figure 3.3: Boxplot per grapheme

To mitigate alignment drift observed in grapheme duration analysis, a silence injection strategy was implemented. In this approach, any grapheme whose duration exceeded its 95th percentile (P95) threshold was considered to have absorbed excessive silence. A corresponding silence token was then inserted to separate the prolonged duration from the grapheme.

This correction serves several purposes:

- It ensures that phoneme durations remain realistic, reflecting natural speech timing.
- It allows silence to be explicitly represented in the dataset, improving the model's understanding of pauses.
- It enhances alignment stability during training, preventing the model from incorrectly associating long pauses with individual graphemes.

4. System Design

This chapter presents the methodology used to design and train the proposed Text-to-Speech (TTS) system. The system follows a non-autoregressive neural speech synthesis framework derived from FastSpeech 2 with architectural enhancements aimed at improving robustness when training on limited Nepali speech datasets.

4.1 System Architecture

The system integrates a Conformer-based encoder-decoder architecture, stochastic variance modeling, and a gated convolutional PostNet for mel-spectrogram refinement. Additionally, the training process incorporates advanced techniques such as teacher-forcing scheduling, Exponential Moving Average (EMA), and cosine learning rate decay to stabilize training.

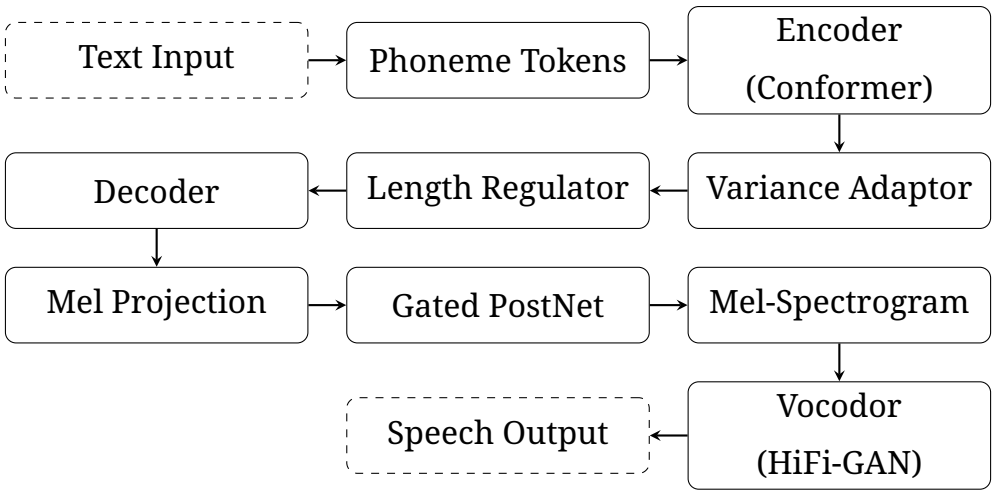

Figure 4.1: Architecture of the Proposed Nepali TTS System.

4.1.1 Phoneme Embedding Layer

The input text is first converted into a sequence of phoneme tokens using a predefined vocabulary mapping tailored for the Nepali language. These discrete tokens are then projected into a continuous vector space through a learnable embedding layer, allowing the model to capture semantic and

phonetic relationships between sounds.

Let the input phoneme token sequence of length N be represented as:

$$X = (x_1, x_2, \ldots, x_N)$$

where each x_i is an index in the phoneme vocabulary.

The embedding layer transforms this sequence into a dense feature matrix E:

$$E = \text{Embedding}(X)$$

where $E \in R^{N \times d_{\text{model}}}$, and d_{model} represents the hidden embedding dimension of the system. These dense vectors serve as the primary input for the subsequent Conformer-based encoder blocks.

4.1.2 Encoder

The encoder is responsible for extracting high-level linguistic and prosodic features from the initial phoneme embeddings. Our model utilizes a Conformer stack, an architecture derived from the Conformer framework [1], designed to overcome the limitations of standard Transformers in modeling local speech patterns.

The Conformer block integrates three fundamental modules to process the input sequence:

- **Multi-Head Self-Attention (MHSA):** Captures long-range contextual dependencies across the entire phoneme sequence.
- **Convolution Module:** Extracts local phonetic patterns and fine-grained acoustic features, which are vital for the tonal nuances of the Nepali language.
- **Feed-Forward Networks (FFN):** Position-wise non-linear transformations applied in a Macaron-style structure to enhance representational power.

Conformer Encoder

The encoder in the proposed system is implemented using the Conformer architecture, which integrates the strengths of convolutional neural net-

works and transformer-based attention mechanisms. This hybrid design allows the model to effectively capture both local phonetic structures and long-range contextual relationships within the input text sequence.

Each Conformer block consists of several key components:

- Feed-forward network module
- Multi-head self-attention module
- Convolution module
- Feed-forward network module
- Layer normalization

The combination of these components enables the encoder to learn robust and expressive linguistic representations, which improves the accuracy of duration prediction and the quality of generated spectrograms.

Let X denote the tokenized input text sequence. The encoder transforms this sequence into a set of hidden representations according to the following relation:

$$H = \text{Encoder}(X)$$

where, X represents the sequence of tokenized input characters, and H denotes the encoded linguistic feature representations produced by the encoder.

These encoded features are then passed to the subsequent modules of the TTS system for alignment estimation and spectrogram generation.

SwiGLU Activation

The feed-forward layers within the Conformer encoder utilize the SwiGLU (Swish-Gated Linear Unit) activation function instead of conventional activation functions. SwiGLU introduces a gating mechanism that enhances the representational capacity of the network while maintaining stable gradient propagation during training.

The SwiGLU activation function is defined as:

$$\text{SwiGLU}(x) = (xW_1) \cdot \sigma(xW_2)$$

where, W_1 and W_2 are learnable weight matrices, and $\sigma(\cdot)$ denotes the sigmoid activation function.

In this formulation, the input is projected into two separate linear transformations. One projection generates the primary feature representation, while the second projection acts as a gating signal through the sigmoid function. The element-wise multiplication between these two components allows the network to dynamically control the flow of information.

The use of SwiGLU improves gradient flow and enhances the expressive power of the model, enabling the encoder to learn richer feature representations. This property is particularly beneficial when training on limited datasets, as it helps the model extract more informative patterns from the available data.

Rotary Positional Embedding (RoPE)

To enhance the model's understanding of sequential order, our model incorporates Rotary Positional Embeddings (RoPE) [11] within the attention mechanism, replacing traditional absolute positional encodings. RoPE encodes relative positional information by rotating the query (q) and key (k) representations in a complex plane.

Given a phoneme at position m and its embedding x, the transformation is defined as:

$$f_{q,k}(x_m, m) = (x_m e^{im\theta}) \tag{4.1}$$

By utilizing a rotation matrix, the attention score between positions m and n becomes a function of their relative distance $m - n$. This methodology allows the encoder to better capture the temporal relationships and rhythmic constraints essential for the prosody of the Nepali language.

4.1.3 Variance Adaptor

Speech is inherently one-to-many; a single text sequence can be realized with various acoustic variations. The Variance Adaptor in our TTS system

addresses this by predicting phoneme-level attributes—duration, pitch, and energy providing the decoder with the necessary prosodic information.

The adaptor consists of four specialized sub-modules:

- **Duration Predictor:** Determines the length (number of mel-frames) for each phoneme. This output is utilized by the Length Regulator to expand the hidden states to match the target spectrogram length.
- **Pitch Predictor (VAE-based):** Employs a Variational Autoencoder (VAE) framework to model the fundamental frequency (F_0). This stochastic approach captures the natural intonation contours of Nepali speech more effectively than deterministic regressors.
- **Energy Predictor:** Predicts the volume and intensity of the speech signal by calculating the L_2-norm of the amplitude of each short-time Fourier transform (STFT) frame.
- **Voiced Predictor:** An auxiliary module that identifies voiced and unvoiced segments, assisting the model in distinguishing between tonal sounds and aspirated consonants or pauses.

The combined outputs of these predictors are summed with the encoder output before being passed to the decoder, ensuring a rich, prosodically-informed representation.

4.1.4 Duration Predictor

The duration predictor estimates the temporal alignment of the speech signal by determining how many mel-spectrogram frames correspond to each input phoneme. Unlike traditional deterministic models that predict a single fixed value, our TTS system adopts a probabilistic approach by modeling duration (d) through a log-normal distribution:

$$d \sim \mathcal{N}(\mu, \sigma^2) \tag{4.2}$$

In this framework, the module predicts two parameters for each phoneme:

- μ: The predicted mean duration in the log domain.
- $\sigma^2 = \exp(\text{logvar})$: The predicted variance, where logvar is the direct output of the network.

The model is optimized using the Negative Log-Likelihood (NLL) loss, which encourages the network to minimize the error while accounting for the inherent uncertainty in phoneme duration:

$$\mathcal{L}_{dur} = \frac{(d-\mu)^2}{2\sigma^2} + \frac{1}{2}\log(\sigma^2)$$

To maintain training stability and prevent the model from converging to degenerate solutions, the predicted variance is constrained within predefined numerical bounds. This stochastic modeling allows the system to generate more natural and varied speech rhythms, which is particularly beneficial for the rhythmic complexities of the Nepali language.

Pitch Predictor (Variational Model)

To capture the complex intonation contours of the Nepali language, the pitch predictor is implemented using a Variational Autoencoder (VAE) formulation. This allows the model to learn a continuous latent space of pitch variations rather than a fixed mapping.

The predictor outputs the parameters of the latent distribution: μ_p and $\log \sigma_p^2$. Pitch samples are generated using the reparameterization trick to ensure the network remains end-to-end differentiable:

$$z = \mu_p + \sigma_p \cdot \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0,1)$$

This stochastic generation enables our TTS system to produce varied and natural pitch contours across different synthesis runs. The training objective for this module combines reconstruction error (MSE) and Kullback-Leibler (KL) divergence to regularize the latent space:

$$\mathcal{L}_{pitch} = \|z - p_{true}\|^2 + \beta \cdot D_{KL}(\mathcal{N}(\mu_p, \sigma_p^2) \| \mathcal{N}(0,1))$$

Voiced Predictor

The voiced predictor is a binary classification module that determines the voicing state (voiced vs. unvoiced) of a phoneme. This is particularly useful for Nepali, which contains many aspirated and unvoiced consonants. The model outputs the voicing probability p_v via a sigmoid activation:

$$p_v = \sigma(Wx + b)$$

The module is trained using Binary Cross-Entropy (BCE) loss:

$$\mathcal{L}_{voiced} = -[y \log(p_v) + (1 - y) \log(1 - p_v)]$$

Energy Predictor

The energy predictor estimates the normalized log-scale intensity of the speech signal. By providing explicit energy information, the model can better reproduce the stress patterns of spoken Nepali. The loss is calculated using Mean Squared Error:

$$\mathcal{L}_{energy} = \|E_{pred} - E_{true}\|^2$$

4.1.5 Length Regulator

The Length Regulator acts as the bridge between the phoneme-level encoder and the frame-level decoder. It expands the hidden representations to match the temporal resolution of the target mel-spectrogram based on the predicted phoneme durations $\{d_1, d_2, \ldots, d_N\}$.

Formally, if H_i represents the hidden state of phoneme i with duration d_i, the regulator repeats the vector d_i times. The resulting frame-level representation H' is defined as:

$$H' = [\underbrace{H_1, \ldots, H_1}_{d_1}, \underbrace{H_2, \ldots, H_2}_{d_2}, \ldots, \underbrace{H_N, \ldots, H_N}_{d_N}]$$

Where $H' \in R^{T \times d_{model}}$ and $T = \sum_{i=1}^{N} d_i$ is the total number of mel frames. This mechanism ensures the decoder receives a sequence aligned with the desired output audio length.

4.1.6 Decoder Architecture

The decoder in in our TTS model is implemented as a Conformer stack, mirroring the encoder's structural complexity. It is responsible for transforming the expanded frame-level features into high-dimensional hidden acoustic representations.

Let the output of the decoder be Y:

$$Y = \text{Decoder}(H')$$

where H' is the expanded sequence from the Length Regulator. By utilizing self-attention and convolutional layers, the decoder effectively captures the temporal relationships between neighboring mel-frames, ensuring smooth transitions and rhythmic consistency. Unlike autoregressive models such as Tacotron 2, this non-autoregressive design facilitates parallel spectrogram generation, significantly reducing inference latency.

4.1.7 Mel Spectrogram Projection

The hidden representation Y is projected into the mel-spectrogram space through a linear transformation layer. This layer maps the d_{model} dimensions into the standard acoustic frequency bins required for audio synthesis:

$$M = W_y Y + b_y$$

where $M \in R^{T \times 80}$ represents the predicted mel-spectrogram (assuming an 80-band resolution), and T is the total number of frames.

4.1.8 PostNet Refinement

To further improve the fidelity of the generated audio, our TTS model incorporates a Gated Convolutional PostNet. While traditional CNN-based PostNets help reduce over-smoothing, this system employs a gated mechanism inspired by WaveNet architectures to provide more precise spectral refinement.

Each refinement block within the PostNet performs the following gated operation:

$$\text{Output} = \tanh(\text{Conv}(x)) \odot \sigma(\text{Conv}(x))$$

where $\odot$ denotes the element-wise Hadamard product and σ represents the sigmoid activation function. The PostNet acts as a residual refiner; the final refined mel-spectrogram $M_{refined}$ is computed as:

$$M_{refined} = M + \Delta M$$

where ΔM is the residual correction predicted by the PostNet. This multi-stage approach ensures that the resulting spectrogram contains the fine-grained spectral details necessary for high-quality Nepali speech reconstruction.

4.1.9 Training Strategy

The training of our TTS model follows a structured, step-based optimization strategy designed to handle the complexities of neural speech synthesis on Nepali datasets. The following advanced techniques are employed to ensure model stability and high-fidelity output.

Teacher Forcing Scheduling

To bridge the gap between training (where ground-truth labels are available) and inference (where the model relies on its own predictions), a scheduled teacher forcing (TF) strategy is implemented. This reduces exposure bias by gradually forcing the model to rely on its internal states. The scheduling follows a step-wise decay as shown in Table.

Training Steps	Teacher Forcing Ratio
0 – 7,000	1.0
7,000 – 13,000	0.8
13,000+	0.5

Table 4.1: Teacher Forcing Scheduling

Learning Rate Scheduling

To ensure smooth convergence and prevent the optimizer from getting stuck in suboptimal local minima, a Warmup Cosine Decay scheduler is

utilized. The learning rate η_t at step t is defined as:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right)$$

where η_{max} and η_{min} are the boundary learning rates and T represents the total number of decay steps. This approach allows for rapid initial learning while ensuring a stable and refined convergence in the later stages of training.

Exponential Moving Average (EMA)

To stabilize the evaluation process and improve the generalization of the model, an Exponential Moving Average (EMA) of the model parameters is maintained throughout the training process. The EMA weights θ_{EMA} are updated at each step using the current model weights θ as follows:

$$\theta_{EMA} = \alpha\theta_{EMA} + (1 - \alpha)\theta$$

In our TTS model, a decay factor of $\alpha = 0.9997$ is used. These smoothed EMA weights are exclusively used during validation and inference to produce more consistent and high-quality speech samples compared to using raw, high-variance training weights.

4.1.10 Loss Functions

The final training objective of our TTS model is a multi-task loss function that jointly optimizes the acoustic reconstruction and the variance predictors. The total loss $\mathcal{L}_{total}$ is defined as the weighted sum of individual components:

$$\mathcal{L}_{total} = \mathcal{L}_{mel} + \mathcal{L}_{postnet} + \mathcal{L}_{dur} + \mathcal{L}_{pitch} + \lambda_{KL}\mathcal{L}_{KL} + \mathcal{L}_{voiced} + \mathcal{L}_{energy}$$

where the constituent terms are defined as follows:

- $\mathcal{L}_{mel}$: Mean Squared Error (MSE) for the initial mel-spectrogram reconstruction.
- $\mathcal{L}_{postnet}$: MSE for the refined mel-spectrogram after the Gated PostNet.

- $\mathcal{L}_{dur}$: Negative Log-Likelihood (NLL) for the stochastic duration predictor.
- $\mathcal{L}_{pitch}$: Reconstruction loss for the VAE-based pitch predictor.
- $\mathcal{L}_{KL}$: Kullback-Leibler divergence for regularizing the pitch latent space.
- $\mathcal{L}_{voiced}$: Binary Cross-Entropy loss for voiced/unvoiced classification.
- $\mathcal{L}_{energy}$: MSE for the phoneme-level energy prediction.

4.1.11 Dataset Preparation

The dataset pipeline is designed to transform raw Nepali speech recordings into a set of aligned acoustic features. The preprocessing workflow involves several signal processing stages to ensure data consistency and training efficiency.

Acoustic Feature Extraction

The raw audio signals undergo the following transformations:

1. **Resampling:** All audio files are resampled to a uniform sampling rate of $22,050$ Hz.
2. **STFT:** A Short-Time Fourier Transform is applied using a 1024-point FFT, a 256-point hop size, and a 1024-point Hanning window.
3. **Mel Projection:** The linear-frequency spectrogram is projected onto an 80-bin mel-scale filterbank.

Prosodic Feature Extraction

To supervise the Variance Adaptor, the following ground-truth features are extracted:

- **Pitch Extraction:** Fundamental frequency (F_0) is extracted using the *Harvest* or *Dio* algorithms from the WORLD vocoder framework. Extracted pitch values are converted to the log scale for modeling:

$$p = \log(F_0)$$

Unvoiced frames are assigned a sentinel value of 0.0 to denote the absence of periodic vibration.

- **Energy Extraction:** The energy E for each frame is calculated as the L_2-norm of the STFT magnitude, subsequently converted to the log domain:

$$E = \log(\|\text{STFT}\|_2)$$

All extracted features (mel-spectrograms, pitch, energy, and durations) are serialized into NumPy arrays to facilitate high-speed data loading during the training phase.

4.1.12 Speech Generation

During inference, the trained model generates speech by converting input text into a mel-spectrogram, which is then transformed into a waveform using a neural vocoder. This process represents the complete text-to-speech synthesis pipeline of the system.

The speech generation procedure consists of the following steps:

- **Text Tokenization:** The input text is first preprocessed and converted into a sequence of tokens based on the predefined vocabulary used during training.

- **Linguistic Feature Encoding:** The token sequence is passed through the encoder, which produces a set of linguistic feature representations that capture the phonetic and contextual information of the input text.

- **Duration Prediction:** The duration predictor estimates the number of spectrogram frames associated with each token. These predicted durations determine the temporal alignment between text tokens and acoustic frames.

- **Mel-Spectrogram Generation:** Using the encoded linguistic features and predicted durations, the flow-based decoder generates the corre-

sponding mel-spectrogram frames, representing the acoustic characteristics of the synthesized speech.

- **Waveform Reconstruction:** The generated mel-spectrogram is converted into a time-domain audio waveform using a HiFi-GAN. The vocoder reconstructs the speech signal while preserving natural sounding acoustic details.

The resulting waveform represents the final synthesized speech, corresponding to the original input text. This process enables the system to produce intelligible and natural-sounding speech from textual input.

5. Evaluation and Result

5.1 Training Performance and Convergence Analysis

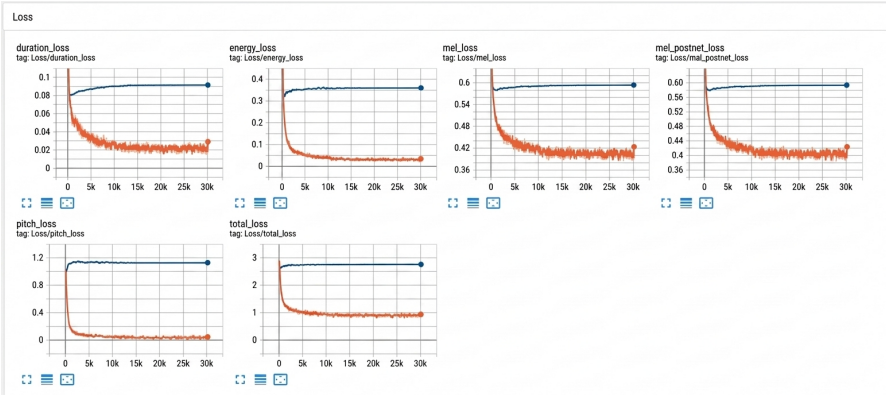

Figure 5.1: Tensorboard loss

The training progress of the nepali TTS model was monitored through a multi-faceted loss objective, as illustrated in the preceding figures. The plots provide a comparative visualization of the training performance (represented in orange) against the validation performance (represented in blue) over a span of 30,000 optimization steps.

The empirical results of the training phase indicate the following key observations:

- **Convergence Characteristics:** The loss curves for all acoustic components including Mel-spectrogram reconstruction, pitch, energy, and duration predictors exhibit a consistent and steady decline. This confirms that the model successfully captured the underlying phonetic and prosodic patterns of the Nepali speech dataset.

- **Generalization Gap:** A persistent divergence is observed between the training and validation trajectories. While the training loss continues to minimize throughout the 30,000 steps, the validation loss reaches a plateau early in the training process, suggesting a limit to the model's predictive accuracy on unseen data.

- **Asymptotic Stability:** The total loss function reaches a state of relative stability after approximately 10,000 steps. The lack of significant further reduction in validation loss beyond this threshold indicates that the model reached its generalization capacity under the current architectural configuration and data volume.

These findings suggest that while the nepali TTS architecture is highly capable of accurately reconstructing the training distribution, future iterations should explore enhanced regularization techniques, such as dropout tuning or data augmentation, to further bridge the gap between training and validation performance.

5.2 Spectrogram Reconstruction Analysis

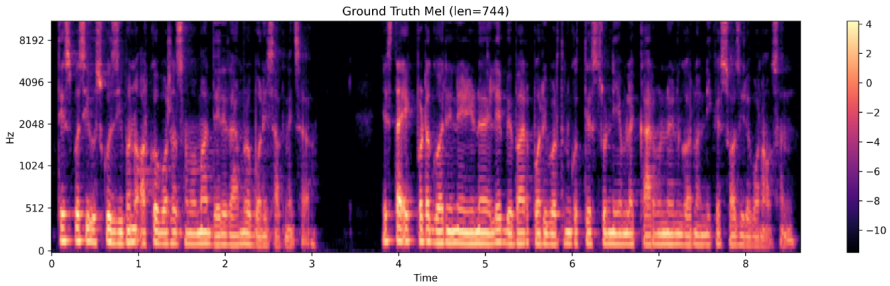

Figure 5.2: Ground Truth Mel-spectrogram

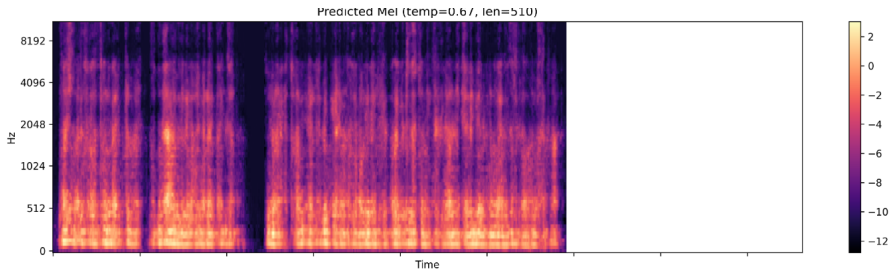

Figure 5.3: Predicted Mel-spectrogram

To evaluate the model's ability to generate high-fidelity speech, a comparative analysis was performed between the Ground Truth Mel-spectrogram and the Predicted Mel-spectrogram generated by our nepali TTS framework. This visual and structural assessment confirms the model's capacity to synthesize complex acoustic features necessary for natural Nepali speech.

- **Spectral Detail:** The predicted spectrogram accurately captures the harmonic structures and formants seen in the ground truth. The model successfully reproduces the temporal spacing (silences) and the frequency distribution essential for intelligible Nepali speech.
- **Acoustic Modeling:** While the predicted mel (temp=0.67) appears slightly smoother than the ground truth, a common characteristic of neural vocoders and acoustic models, it retains the vital energy peaks required for clear pronunciation.

This minor smoothing effect acts as a regularizer, often resulting in reduced background noise in the final synthesized waveform while maintaining the spectral details necessary for high-fidelity audio reconstruction. The alignment of these spectral peaks confirms that the model has successfully learned the mapping from phoneme sequences to detailed acoustic representations.

5.3 Subjective Evaluation: Mean Opinion Score (MOS)

To evaluate the perceptual quality and naturalness of the synthesized Nepali speech, a subjective listening test was conducted. This evaluation aims to bridge the gap between objective loss metrics and actual human perception, providing a more holistic view of the system's performance.

The evaluation followed the standard Absolute Category Rating (ACR) procedure. A cohort of 10 participants, all native Nepali speakers, was recruited for the study. Each participant was presented with 5 randomly selected audio samples generated by our model. Listeners were tasked with rating each sample based on its naturalness, clarity, and prosody using a 5-point Likert scale.

Table 5.1: MOS Rating Scale and Quality Descriptions

Score	Quality	Description
5	Excellent	Imperceptible difference from human speech
4	Good	Clear, but with minor synthetic artifacts
3	Fair	Perceptibly synthetic but completely intelligible
2	Poor	Significant distortion or unnatural prosody
1	Bad	Very difficult to understand

5.3.1 Results

The results of the subjective evaluation are summarized in Table 5.2. The model achieved a competitive average score, demonstrating a high level of intelligibility and phonetic accuracy for the Nepali language.

Table 5.2: Subjective Evaluation Results Summary

Metric	Value
Number of Evaluators	10
Samples per Evaluator	5
Total Evaluations	50
Average MOS	**3.5**

An average **MOS of 3.5** indicates that the model's output is perceived as falling within the *"Fair to Good"* range. In the specific context of Nepali TTS, which often faces challenges due to a lack of high-quality, large-scale open-source datasets compared to English, a score of 3.5 represents a robust baseline.

This performance suggests that while the synthesized speech is clearly intelligible and adheres to the correct phonetic structure of the Nepali language, there remains an opportunity for further refinement.

6. Conclusions

The development of this Nepali Text-to-Speech (TTS) system successfully demonstrates the application of deep learning for low-resource linguistic synthesis. Over a training period of 30,000 steps, the model achieved stable convergence across all primary loss functions, including duration, pitch, and energy. Objective analysis through Mel-spectrogram comparison confirms that the system effectively reconstructs the essential harmonic structures and temporal patterns required for clear Nepali speech.

Subjective evaluation by native speakers further validates these results, with the model earning a Mean Opinion Score (MOS) of 3.5. This score indicates that the synthesized output is consistently intelligible and reaches a ”Good” quality threshold for practical use. While the performance gap between training and validation metrics suggests room for further refinement in prosodic naturalness, this project provides a robust foundation for future Nepali speech synthesis research and accessibility tools.

7. Limitations and Future enhancement

7.1 Limitations

Despite promising mid-term results, several limitations persist:

- **Data Scarcity:** The model was trained on a relatively small dataset of 4,500 audio samples, which limits its ability to generalize across diverse phonetic contexts and speaker variations.
- **Lack of Prosody Modeling:** The absence of a dedicated prosody encoder results in speech that can feel monotonous or robotic, as the system cannot yet capture complex intonation patterns.
- **Language-Specific Challenges:** Nepali's tonal nuances and script complexities occasionally lead to alignment errors in MFA, affecting synthesis quality.
- **Optimization Constraints:** Due to resource limits, exhaustive hyperparameter tuning was not conducted, leaving the model at a suboptimal configuration for the specific Nepali phonetic space.
- **Generalization Gap:** As seen in the training logs, there is a notable gap between training and validation loss, suggesting the model is prone to overfitting on the limited available data.

7.2 Future Enhancement

To overcome these limitations, future work will include:

- **Dataset Expansion:** Prioritize the collection and cleaning of a larger, multi-speaker Nepali corpus to improve the model's robustness and naturalness, moving beyond the current limitation of 4,500 samples.

- **Prosody and Style Integration:** Incorporate a dedicated prosody encoder or Global Style Tokens (GST) to allow the model to learn and reproduce expressive emotional states, rhythmic variations, and tonal nuances inherent in Nepali speech.
- **Advanced Linguistic Front-end:** Develop a more sophisticated text-to-phoneme pipeline specifically tailored to handle the complexities of the Devanagari script, including conjunct characters and schwa deletion rules.
- **Hyperparameter Optimization:** Utilize automated search techniques, such as Bayesian optimization, to fine-tune critical parameters including learning rates (α), batch sizes, and layer dimensions to minimize the generalization gap observed in current loss curves.
- **Vocoder Upgrade:** Experiment with high-fidelity vocoders such as HiFi-GAN or BigVGAN to reduce the spectral smoothing seen in the current predicted mel-spectrograms and produce crisper, more human-like audio output.

References

[1] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition, 2020.

[2] Yi Ren, Chenxu Hu, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. Fastspeech 2: Fast and high-quality end-to-end text to speech, 2022.

[3] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis. *CoRR*, abs/2010.05646, 2020.

[4] F. E. Idachaba. A simplified overview of text-to-speech synthesis. *International Journal of Electrical and Electronics Research*, 4(1):1–6, 2016.

[5] S. R. Bajracharya, A. Pradhan, B. B. Gautam, and A. Gautam. Development of a text-to-speech system for nepali language: A case study. *Gipan*, 2:1–12, 2021.

[6] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. W. Senior, and K. Kavukcuoglu. Wavenet: A generative model for raw audio. In *Proc. 9th ISCA Speech Synthesis Workshop*, pages 125–125, Sunnyvale, CA, USA, 2016. ISCA.

[7] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. High fidelity neural audio compression. *CoRR*, abs/2210.13438, 2022.

[8] B. Chettri and K. B. Shah. Nepali text to speech synthesis system using esnola method of concatenation. *International Journal of Computer Applications*, 62(2):24–28, 2013.

[9] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T. Liu. Fastspeech: Fast, robust and controllable text to speech. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[10] Subodh Poudel. Youtube channel: Subodh poudel. `https://www.youtube.com/@subodhpoudel1`, 2025. Accessed on August 14, 2025.

[11] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021.